

PERFORMANCE ENGINEERING

Lab 4

Jort Oosterveld S5545781 & Sergiu Preda S6778542

Group 11

September 30, 2026

1 SQUARED SUMS

1.1 SYSTEM

All measurements were taken on the system in Table 1. The benchmark was pinned to one core with `taskset`.

Table 1: System used for the measurements.

CPU	AMD Ryzen 7 5825U (Zen 3, family 25, model 80)
Cores / threads	8 / 16
Frequency	max. 4.55 GHz (boost enabled), 4.50 GHz assumed for cycle counts
Caches	L1d 32 KiB and L2 512 KiB per core, L3 16 MiB shared
SIMD registers	16 (AVX2, FMA3)
OS	Linux 7.0.0-31-generic
Governor	performance (amd-pstate-epp)
Compiler	g++ (GCC) 15.2.1
Flags	-std=c++17 -O2 -march=native -fno-tree-vectorize -ffp-contract=off

The microarchitecture parameters of Zen 3 that matter for this exercise are listed in Table 2. They are taken from the AMD Software Optimization Guide for family 19h processors and the instruction tables of uops.info.

Table 2: Relevant microarchitecture parameters of Zen 3 (scalar double precision).

Instruction	Latency (cycles)	Throughput (per cycle)	Execution units
vaddsd	3	2	FP2, FP3
vmulsd	3	2	FP0, FP1
vfmadd231sd	4	2	FP0, FP1
Dispatch width: 6 macro-ops per cycle			

1.2 BENCHMARK

The loop of `sqsun` is unrolled by a factor U with one accumulator per unrolled statement, which breaks the dependency for the one accumulator. The complete benchmark program `loop_unrolling.cpp` can be found in Listing 10 in Appendix A.

Two compiler flags are needed to measure what is intended. `-fno-tree-vectorize` keeps the loop scalar, and `-ffp-contract=off` keeps the separate multiply and add, otherwise GCC fuses them into an FMA by itself.

As in the assignment, $n = 8192$ and `sqsum` is called 10000 times. This batch of 10000 calls is timed with `clock_gettime(CLOCK_MONOTONIC_RAW)` and repeated 30 times per unrolling factor. Only the minimum (best case) of the 30 repetitions is reported, since disturbances by the operating system can only make a run slower.

Hardware cycle counters are not accessible on this system (`perf_event_paranoid = 4`). The time per element is therefore converted to cycles with an assumed constant core frequency of 4.50 GHz, the boost clock the core runs at with the `performance` governor while the benchmark is pinned to it. The cycle counts are therefore only as accurate as this assumption, the times are what was measured.

1.3 RESULTS

Table 3 shows the results (`loop_unrolling.csv`).

Table 3: Best case of 30 repetitions for the squared sum with unrolling factor U , $n = 8192$, 10000 calls per repetition. The speed-up is relative to the loop without unrolling.

U	$\mu\text{s} / \text{call}$	ns / element	cycles / element	speed-up
1	5.49	0.6705	3.017	1.00
2	2.78	0.3388	1.525	1.98
3	1.88	0.2297	1.034	2.92
4	1.55	0.1888	0.850	3.55
5	1.25	0.1532	0.689	4.38
6	1.23	0.1498	0.674	4.48
7	1.20	0.1464	0.659	4.58
8	1.07	0.1306	0.587	5.13
9	1.14	0.1390	0.625	4.82
10	1.12	0.1365	0.614	4.91
11	1.06	0.1300	0.585	5.16
12	1.15	0.1404	0.632	4.78
14	1.05	0.1280	0.576	5.24
16	1.62	0.1982	0.892	3.38

1.4 A) OPTIMAL UNROLLING FACTOR

Without unrolling, every addition has to wait for the previous one, so one element costs the latency of the loop-carried operation. Only the addition is part of this dependency chain, the multiplication depends on the loaded value alone. The measured 3.02 cycles per element for $U = 1$ match the latency of 3 cycles of `vaddsd`.

With U accumulators there are U independent chains, so the cost drops to $3/U$ cycles per element until the throughput of the core becomes the limit. Zen 3 executes 2 additions and 2 multiplications per cycle, which is 0.5 cycles per element. The chains stop being the bottleneck at

$$U_{\text{opt}} = \text{latency} \times \text{throughput} = 3 \times 2 = 6.$$

The measurements follow $3/U$ up to $U = 3$ and then flatten out. after $U = 6$, there is no systematic gain anymore. All factors from 6 to 14 lie between 0.58 and 0.67 cycles per element. This agrees with the expected factor of 6, which is 4.5 times faster than the original loop. The theoretical 0.5 cycles per element is not reached. A likely reason is the dispatch width: the loop needs 3 macro-ops per element, a load, multiply and add. So 0.5 cycles per element would already use all 6 macro-ops per cycle that Zen 3 can dispatch, and the loop overhead comes on top. This also explains why $U = 8$ is clearly faster than $U = 6$ with 0.587 cycles per element. There is less

loop overhead (shown in Listings 1 and 2) when unrolling is done with a power of two, which gives an estimate of $(3 \cdot 8 + 2)/(6 \cdot 8) = 0.54$ cycles per element against $(3 \cdot 6 + 4)/(6 \cdot 6) = 0.61$ for $U = 6$. With the same estimate the overhead of 4 macro-ops is spread over more elements for larger factors, so $U = 11$ and $U = 14$ are as fast as $U = 8$. The differences between the factors 8 to 14 can be attributed to variations between runs.

```

1  leal    11(%rcx), %edx
2  addq    $48, %rax
3  addl    $6, %ecx
4  cmpl    %edx, %esi
5  jg      .L51

```

Listing 1: Loop overhead of compiled assembler with u=6

```

1  addq    $64, %rax
2  cmpq    %rax, %rdx
3  jne     .L80

```

Listing 2: Loop overhead of compiled assembler with u=8

1.5 B) FUSED-MULTIPLY-ADD

With FMA, the multiplication and addition are one instruction, so there are 2 macro-ops per element instead of 3. The drawback is that the whole FMA is now part of the loop-carried dependency chain, so its latency of 4 cycles replaces the 3 cycles of the addition. Without unrolling, FMA is therefore expected to be about $\frac{1}{3}$ slower.

The throughput of FMA on Zen 3 is 2 per cycle, the same as for the separate multiply and add. But, more accumulators are needed to reach it, $4 \times 2 = 8$ instead of 6. For every unrolling factor below 8 the dependency chain of the FMA is still the bottleneck, so FMA is slower.

FMA can only be better when the loop is unrolled by at least 8. The execution units then allow the same 0.5 cycles per element for both variants, but the multiply and add loop needs all 6 macro ops per cycle of the dispatch width to get there, which is the likely reason that it only reached 0.58 cycles per element in the measurements. With 2 macro ops per element FMA needs only 4 per cycle, so it could come closer to the limit.

The relevant microarchitecture parameters are:

- the latency of the addition compared to the latency of the FMA, since this is the length of the loop-carried dependency chain,
- the throughput (number of execution units) of addition, multiplication and FMA, which sets the limit once the dependency is broken,
- the pipeline (dispatch) width in relation to the number of macro-ops per element,
- the number of registers, which limits the number of accumulators (15 on this system).

2 AUTO-VECTORIZATION

This exercise was done on the system of Sergiu Preda (Intel Core i7-12700H, GCC 11.4.0 under WSL, see Section 3). The processor supports AVX2 with FMA, so one 256-bit `ymm` register holds 4 doubles. AVX-512 is not available on it.

The matrix is stored row-major. The implementation from the previous lab is shown in Listing 3, the complete program with the timing code is given in Appendix B.

```

1 void matvec(const double * restrict A,
2             const double * restrict x,
3             double * restrict y,

```

```

4         int N)
5     {
6         for (int i = 0; i < N; i++) {
7             double sum = 0.0;
8
9             for (int j = 0; j < N; j++) {
10                sum += A[(size_t)i * N + j] * x[j];
11            }
12
13            y[i] = sum;
14        }
15    }

```

Listing 3: Matrix-vector multiplication (`matvec.c`).

The inner loop over j is the candidate for vectorization: it reads consecutive elements of one row of A and of $\vec{x}$, so the compiler can load 4 elements at once. The outer loop over i is not a candidate, its iterations are N elements apart in A .

2.1 A) DID THE COMPILER VECTORIZE?

COMPILER REPORT

The program was compiled with the following flags to enable the vectorizer and AVX2, and print the vectorizer report.

```

1 gcc -O3 -march=native -fopt-info-vec-optimized -fopt-info-vec-missed \
2     matvec.c -o matvec

```

The relevant lines of the report are given below, the full report is in Appendix B. Line 13 is the inner loop over j , line 10 the outer loop over i .

```

1 matvec.c:10:23: missed: couldn't vectorize loop
2 matvec.c:14:21: missed: not vectorized: complicated access pattern.
3 matvec.c:13:27: optimized: loop vectorized using 32 byte vectors
4 matvec.c:13:27: optimized: loop vectorized using 16 byte vectors

```

So the inner loop is vectorized with 32-byte (AVX2) vectors, plus a 16-byte version for the remaining elements when N is not a multiple of 4. The outer loop is not vectorized, which is expected.

ASSEMBLER

The assembler was generated with

```

1 gcc -O3 -march=native -S -masm=intel matvec.c -o matvec.s

```

Listing 4 shows the inner loop, the whole function is in Appendix B.

```

1 .L7:
2     vmovupd    ymm5, YMMWORD PTR [rcx+rax]
3     vmulpd     ymm1, ymm5, YMMWORD PTR [rsi+rax]
4     add        rax, 32
5     vaddsd     xmm0, xmm0, xmm1
6     vunpckhpd  xmm2, xmm1, xmm1
7     vextractf128 xmm1, ymm1, 0x1
8     vaddsd     xmm0, xmm0, xmm2
9     vaddsd     xmm0, xmm0, xmm1
10    vunpckhpd  xmm1, xmm1, xmm1
11    vaddsd     xmm0, xmm0, xmm1
12    cmp        rax, r8
13    jne        .L7

```

Listing 4: Inner loop of `matvec` with `-O3 -march=native`.

We can see exactly what the report has told us before. Every iteration loads 4 elements of the row (`vmovupd` into `ymm5`) and multiplies them with 4 elements of $\vec{x}$ in one packed instruction (`vmulpd`). The addition is not packed, however: the 4 products are added to the scalar sum in `xmm0` one after the other with 4 `vaddsd`.

2.2 B) CHANGES NEEDED FOR AUTO-VECTORIZATION

No change to the source was needed to get the inner loop vectorized, the code was already suitable for it, because The inner loop runs over the contiguous rows. The sum was already a local variable instead of `y[i]`, so the inner loop contains no stores. The `restrict` qualifiers make no difference for this loop, aliasing only matters when a store could overlap a later load. We checked this by removing `restrict`, the report and the generated code remained the same.

The one change that does make a difference is allowing the compiler to reorder the additions.

```
1 gcc -O3 -march=native -ffast-math matvec.c -o matvec_vec
```

With `-ffast-math` the inner loop becomes Listing 5: one packed FMA per 4 elements into the vector accumulator `ymm0`, which holds 4 partial sums that are added together once after the loop. The result can differ from the original in the last bits due to the order of rounding changing.

```
1 vmovupd    ymm3, YMMWORD PTR [rcx+rax]
2 vfmadd231pd ymm0, ymm3, YMMWORD PTR [rsi+rax]
3 add       rax, 32
4 cmp       rax, r8
5 jne       .L7
```

Listing 5: Inner loop of `matvec` with `-O3 -march=native -ffast-math` (from `objdump`).

3 HARDWARE INFO

3.1 JORT OOSTERVELD

Type of computer: Laptop
Operating system: Ubuntu 24.04.4 LTS
Kernel version: Linux 7.0.0-31-generic
Hardware Model: Acer Aspire AG15-42P
Graphics: AMD Radeon Graphics

```
1 description: Computer
2 width: 64 bits
3 capabilities: smp vsyscall32
4 *-core
5     description: Motherboard
6     physical id: 0
7     *-memory
8         description: System memory
9         physical id: 0
10        size: 15GiB
11    *-cpu
12        product: AMD Ryzen 7 5825U with Radeon Graphics
13        vendor: Advanced Micro Devices [AMD]
14        physical id: 1
15        bus info: cpu@0
16        version: 25.80.0
17        size: 3629MHz
18        capacity: 4547MHz
19        width: 64 bits
```

Listing 6: From lshw

```
1 $ lscpu --cache
2
3 NAME ONE-SIZE ALL-SIZE WAYS TYPE LEVEL SETS PHY-LINE COHERENCY-SIZE
4 L1d 32K 256K 8 Data 1 64 1 64
5 L1i 32K 256K 8 Instruction 1 64 1 64
6 L2 512K 4M 8 Unified 2 1024 1 64
7 L3 16M 16M 16 Unified 3 16384 1 64
```

Listing 7: From lscpu

3.2 SERGIU PREDA

Type of computer: Laptop
Operating system: Windows 11 with WSL Ubuntu 22.04 LTS
Kernel version: Linux 6.18.33.2-microsoft-standard-WSL2
Hardware Model: ASUS ROG TUF F15
Memory: 7.6GiB

```
1 Model name: 12th Gen Intel(R) Core(TM) i7-12700H
2 Flags: ... avx2 ...
```

Listing 8: From lscpu

```
1 $ lscpu --cache
2 NAME ONE-SIZE ALL-SIZE WAYS TYPE LEVEL SETS PHY-LINE COHERENCY-SIZE
3 L1d 48K 480K 12 Data 1 64 1 64
4 L1i 32K 320K 8 Instruction 1 64 1 64
5 L2 1.3M 12.5M 10 Unified 2 2048 1 64
6 L3 24M 24M 12 Unified 3 32768 1 64
```

Listing 9: From lscpu --cache

For additional microarchitecture information, the following sources were used:

- uops.info instruction latency and throughput database: <https://uops.info/>
- Agner Fog instruction tables: https://www.agner.org/optimize/instruction_tables.pdf

A EXERCISE 1: SQUARED SUM BENCHMARK

Listing 10 is the complete benchmark program `loop_unrolling.cpp`. It is compiled and run with `run_benchmark.sh`, which pins it to one core with `taskset` and writes `loop_unrolling.csv`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 // timer helper
6 long long now_ns()
7 {
8     struct timespec ts;
9     clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
10    return ts.tv_sec * 1000000000LL + ts.tv_nsec;
11 }
12
13 // Squared sum, unrolled by a factor U with one accumulator per unroll
14 template <int U>
15 __attribute__((noinline)) // ask the compiler not to perform inlining to keep
16    the unroll factor the only change
17 double sqsum(double *__restrict a, int n)
18 {
19     double s[U] = {};
20     int i = 0;
21
22     for (; i + U - 1 < n; i += U) {
23         // unroll inner loop fully, because we want all U accumulators written
24         // here
25         #pragma GCC unroll 16 // max unroll we test
26         for (int k = 0; k < U; k++) {
27             s[k] += a[i + k] * a[i + k];
28         }
29
30         // remainder when n is not a multiple of U
31         for (; i < n; i++) {
32             s[0] += a[i] * a[i];
33         }
34
35         double total = 0.0;
36         for (int k = 0; k < U; k++){
37             total += s[k];
38         }
39
40         return total;
41     }
42 }
43
44 struct Kernel {
45     int unroll;
46     double (*fn)(double *__restrict, int);
47 };
48
49 #define KERNEL(U) {U, sqsum<U>},
50
51 // called through this table, so the compiler cannot inline the kernels or
52 // hoist the calls out of the timing loop
53 const Kernel kernels[] = {
54     KERNEL(1) KERNEL(2) KERNEL(3) KERNEL(4) KERNEL(5) KERNEL(6) KERNEL(7)
55     KERNEL(8) KERNEL(9) KERNEL(10) KERNEL(11) KERNEL(12) KERNEL(14) KERNEL(16)
56 };
57
58 double sum;
```

```

57
58 int main()
59 {
60     int n = 8192;
61     int calls = 10000;
62     int reps = 30;
63
64     // Multiples of 0.25 so no floating point inaccuracies
65     double *a = (double *) calloc(n, sizeof(double));
66     double expected = 0.0;
67     for (int i = 0; i < n; i++) {
68         a[i] = (i % 16) * 0.25;
69         expected += a[i] * a[i];
70     }
71
72     printf("unroll,min_ns,ns_per_element,cycles_per_element\n");
73
74     for (const Kernel& kernel : kernels) {
75         // assume max clock speed for Jort's system
76         double core_ghz = 4.50;
77
78         // best case of 'reps' batches of 'calls' calls
79         long long min_ns = 0;
80         for (int rep = 0; rep < reps; rep++) {
81             long long start = now_ns();
82             for (int i = 0; i < calls; i++) {
83                 sum = kernel.fn(a, n);
84             }
85             long long elapsed = now_ns() - start;
86
87             if (rep == 0 || elapsed < min_ns) {
88                 min_ns = elapsed;
89             }
90         }
91
92         if (sum != expected) {
93             fprintf(stderr, "Error: U=%d returned %f, expected %f\n",
94                     kernel.unroll, sum, expected);
95             return 1;
96         }
97
98         double ns_per_element = (double) min_ns / n / calls;
99         printf("%d,%lld,%.4f,%.3f\n", kernel.unroll, min_ns, ns_per_element,
100              ns_per_element * core_ghz);
101     }
102
103     free(a);
104     return 0;
105 }

```

Listing 10: Complete benchmark program loop_unrolling.cpp.

B EXERCISE 2: MATRIX-VECTOR PROGRAM, COMPILER REPORT AND ASSEMBLER

Listing 11 is the complete program `matvec.c` with the timing code. The vectorization report in Listing 12 was produced before the include of `time.h` was added, so its line numbers are one lower than in Listing 11. Listing 13 is the `matvec` function from the assembler output `matvec.s` (`-O3 -march=native -S -masm=intel`).

```

1 #include <stdio.h>

```

```

2 #include <stdlib.h>
3 #include <stddef.h>
4 #include <time.h>
5
6 void matvec(const double * restrict A,
7             const double * restrict x,
8             double * restrict y,
9             int N)
10 {
11     for (int i = 0; i < N; i++) {
12         double sum = 0.0;
13
14         for (int j = 0; j < N; j++) {
15             sum += A[(size_t)i * N + j] * x[j];
16         }
17
18         y[i] = sum;
19     }
20 }
21
22 int main() {
23     int N = 2048; // matrix size large enough to benefit from vectorization
24
25     // Allocate memory for the N x N matrix and the two vectors
26     double *A = (double *)malloc((size_t)N * N * sizeof(double));
27     double *x = (double *)malloc((size_t)N * sizeof(double));
28     double *y = (double *)malloc((size_t)N * sizeof(double));
29
30     if (A == NULL || x == NULL || y == NULL) {
31         fprintf(stderr, "Memory allocation failed.\n");
32         return 1;
33     }
34
35     // prevent dead-code elimination
36     for (size_t i = 0; i < (size_t)N * N; i++) {
37         A[i] = 1.0;
38     }
39     for (int i = 0; i < N; i++) {
40         x[i] = 2.0;
41         y[i] = 0.0;
42     }
43
44     // multiplication
45     struct timespec start, end;
46     clock_gettime(CLOCK_MONOTONIC, &start);
47
48     // Execute the multiplication
49     matvec(A, x, y, N);
50
51     clock_gettime(CLOCK_MONOTONIC, &end);
52     double time_taken = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.
53 tv_nsec) * 1e-9;
54
55     printf("Execution time: %f seconds\n", time_taken);
56
57     printf("Result y[0] = %f\n", y[0]);
58
59     // Clean up memory
60     free(A);
61     free(x);
62     free(y);
63
64     return 0;

```

Listing 11: Complete program `matvec.c`.

```

1 $ gcc -O3 -march=native -fopt-info-vec-optimized -fopt-info-vec-missed matvec.c
  -o matvec
2 matvec.c:10:23: missed: couldn't vectorize loop
3 matvec.c:14:21: missed: not vectorized: complicated access pattern.
4 matvec.c:13:27: optimized: loop vectorized using 32 byte vectors
5 matvec.c:13:27: optimized: loop vectorized using 16 byte vectors
6 matvec.c:10:23: missed: couldn't vectorize loop
7 matvec.c:14:21: missed: not vectorized: complicated access pattern.
8 matvec.c:13:27: optimized: loop vectorized using 32 byte vectors
9 matvec.c:38:23: optimized: loop vectorized using 32 byte vectors
10 matvec.c:35:26: optimized: loop vectorized using 32 byte vectors
11 matvec.c:25:27: missed: statement clobbers memory: A_21 = malloc (33554432);
12 matvec.c:26:27: missed: statement clobbers memory: x_23 = malloc (16384);
13 matvec.c:27:27: missed: statement clobbers memory: y_25 = malloc (16384);
14 matvec.c:40:14: missed: statement clobbers memory: __builtin_memset (y_25, 0,
    16384);
15 /usr/include/x86_64-linux-gnu/bits/stdio2.h:112:10: missed: statement clobbers
    memory: __printf_chk (1, "Result y[0] = %f\n", _12);
16 matvec.c:49:5: missed: statement clobbers memory: free (A_21);
17 matvec.c:50:5: missed: statement clobbers memory: free (x_23);
18 matvec.c:51:5: missed: statement clobbers memory: free (y_25);
19 /usr/include/x86_64-linux-gnu/bits/stdio2.h:105:10: missed: statement clobbers
    memory: __builtin_fwrite ("Memory allocation failed.\n", 1, 26, stderr.0_4);

```

Listing 12: Full GCC vectorization report (`-O3 -march=native`).

```

1 matvec:
2 .LFB39:
3   .cfi_startproc
4   endbr64
5   test    ecx, ecx
6   jle     .L19
7   push    rbp
8   .cfi_def_cfa_offset 16
9   .cfi_offset 6, -16
10  lea     eax, -1[rcx]
11  mov     r9, rdx
12  mov     rbp, rsp
13  .cfi_def_cfa_register 6
14  push    r15
15  vxorpd  xmm4, xmm4, xmm4
16  .cfi_offset 15, -24
17  movsx   r15, ecx
18  push    r14
19  .cfi_offset 14, -32
20  lea     r14, 8[rdx+rax*8]
21  xor     edx, edx
22  push    r13
23  .cfi_offset 13, -40
24  mov     r13, rax
25  mov     eax, ecx
26  push    r12
27  shr     eax, 2
28  lea     r8d, -1[rax]
29  inc     r8
30  .cfi_offset 12, -48
31  mov     r12d, ecx
32  push    rbx
33  .cfi_offset 3, -56
34  sal     r8, 5

```

```

35     mov ebx, ecx
36     and r12d, -4
37     .p2align 4,,10
38     .p2align 3
39 .L3:
40     cmp r13d, 2
41     jbe .L10
42     lea rcx, [rdi+rdx*8]
43     xor eax, eax
44     vmovsd xmm0, xmm4, xmm4
45     .p2align 4,,10
46     .p2align 3
47 .L7:
48     vmovupd ymm5, YMMWORD PTR [rcx+rax]
49     vmulpd ymm1, ymm5, YMMWORD PTR [rsi+rax]
50     add rax, 32
51     vaddsd xmm0, xmm0, xmm1
52     vunpckhpd xmm2, xmm1, xmm1
53     vextractf128 xmm1, ymm1, 0x1
54     vaddsd xmm0, xmm0, xmm2
55     vaddsd xmm0, xmm0, xmm1
56     vunpckhpd xmm1, xmm1, xmm1
57     vaddsd xmm0, xmm0, xmm1
58     cmp rax, r8
59     jne .L7
60     cmp r12d, ebx
61     je .L4
62     mov ecx, r12d
63     mov eax, r12d
64 .L9:
65     mov r10d, ebx
66     sub r10d, ecx
67     cmp r10d, 1
68     je .L5
69     vmovupd xmm1, XMMWORD PTR [rsi+rcx*8]
70     lea r11, [rcx+rdx]
71     vmulpd xmm1, xmm1, XMMWORD PTR [rdi+r11*8]
72     mov ecx, r10d
73     and ecx, -2
74     add eax, ecx
75     vaddsd xmm0, xmm0, xmm1
76     vunpckhpd xmm1, xmm1, xmm1
77     vaddsd xmm0, xmm1, xmm0
78     cmp r10d, ecx
79     je .L4
80 .L5:
81     cdqe
82     lea rcx, [rax+rdx]
83     vmovsd xmm6, QWORD PTR [rdi+rcx*8]
84     vfmadd231sd xmm0, xmm6, QWORD PTR [rsi+rax*8]
85 .L4:
86     vmovsd QWORD PTR [r9], xmm0
87     add r9, 8
88     add rdx, r15
89     cmp r14, r9
90     jne .L3
91     vzeroupper
92     pop rbx
93     pop r12
94     pop r13
95     pop r14
96     pop r15
97     pop rbp

```

```

98  .cfi_remember_state
99  .cfi_def_cfa 7, 8
100 ret
101 .p2align 4,,10
102 .p2align 3
103 .L10:
104  .cfi_restore_state
105  xor ecx, ecx
106  xor eax, eax
107  vmovsd xmm0, xmm4, xmm4
108  jmp .L9
109 .L19:
110  .cfi_def_cfa 7, 8
111  .cfi_restore 3
112  .cfi_restore 6
113  .cfi_restore 12
114  .cfi_restore 13
115  .cfi_restore 14
116  .cfi_restore 15
117  ret
118  .cfi_endproc
119 .LFE39:
120  .size matvec, .-matvec

```

Listing 13: The `matvec` function in `matvec.s` (`-O3 -march=native`).